

Nesterov Accelerated Gradient (NAG) Explained

Optimizers in Deep Learning — Part 3 | Notes based on the CampusX video

1. What is NAG?

NAG stands for **Nesterov Accelerated Gradient**. It is an optimization technique built as a small but clever upgrade on top of Momentum Optimization, and in most cases it performs better than plain Momentum.

Recall the problem Momentum was designed to solve: vanilla (mini-batch) gradient descent follows a slow, zig-zag path toward the minimum. Momentum speeds this up by using an exponentially weighted moving average of past gradients, so the optimizer behaves like a ball rolling downhill, building up speed as it moves.

However, this "rolling ball" behavior has a side effect: the ball can pick up so much speed that it **overshoots** the minimum, crosses over to the other side, comes back, overshoots again (a bit less this time), and so on — oscillating back and forth before finally settling down. These oscillations cost extra time, especially on more complex (non-convex) loss surfaces.

NAG tackles exactly this oscillation problem. The core idea: instead of calculating the gradient at your current position, you first take a step using only the accumulated momentum (a "look-ahead" step), and *then* calculate the gradient at that look-ahead position. This small change lets the optimizer "peek ahead," anticipate that it's approaching the minimum, and slow down earlier — reducing overshoot and damping the oscillations.

2. Momentum vs. NAG: What the Animations Show

The video demonstrates this using a simple linear regression problem (mean squared error loss), visualized as a 3D error surface and its 2D contour plot. Starting from a random point, the optimizer must travel down to the true minimum (the optimal slope and intercept).

- **Batch Gradient Descent:** Takes a slow, direct path but needs around 25–30 epochs to converge on this dataset.
- **Momentum:** Behaves like a ball rolling down a hill — it overshoots the minimum, crosses back, overshoots again with a smaller swing, and gradually settles. With $\beta = 0.9$, it actually took *longer* overall (around 50 epochs) on this example because the oscillations themselves ate up time, even though it reached the *vicinity* of the minimum very fast (within about 3 epochs).
- **Reducing β :** Lowering the momentum factor (e.g. to 0.5) reduces the oscillations and brings convergence down to around 38–39 epochs — much better than the 50 epochs at $\beta = 0.9$, though still slower than the few epochs it took to merely approach the minimum.
- **NAG:** With the look-ahead trick applied, oscillations are visibly reduced compared to Momentum at the same β , and the optimizer reaches the minimum faster overall.

Key takeaway: Momentum's biggest strength (fast initial progress) comes with a real cost (oscillation near the minimum). On simple convex problems you can manage this by tuning β down. But on more complex, non-convex loss surfaces, these oscillations become a bigger problem and cost significant extra convergence time. NAG was designed specifically to address this.

3. Mathematical Intuition: Recap of Momentum

The math behind NAG is simple if you already know the math behind Momentum, since NAG is just one small mathematical tweak on the Momentum update rule. So let's recap Momentum first.

In Momentum with SGD, the update rule is:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \mathbf{v}_t$$

where the velocity term is:

$$\mathbf{v}_t = \beta \cdot \mathbf{v}_{t-1} + \alpha \cdot \nabla L(\mathbf{w}_{t-1})$$

Substituting the velocity term into the weight-update equation shows that the update has two components: one coming from the **accumulated past velocity** (the momentum term), and one coming from the **current gradient** at the present position. In plain gradient descent, only the second term existed. Momentum adds the first term, which is what lets it take longer, faster jumps in a consistent direction.

4. Mathematical Formulation of NAG

In Momentum, at any given point you calculate **two things together**: the momentum (from past velocity) and the gradient (at your current position) — and you take both jumps at once.

NAG introduces just one small difference: instead of calculating the gradient at your current position, you first take the momentum jump alone to arrive at a **look-ahead position**, and *then* you calculate the gradient at that look-ahead position. That gradient is what determines your next move.

Step 1 — compute the look-ahead position using momentum only:

$$\mathbf{w}_{\text{look-ahead}} = \mathbf{w}_{t-1} - \beta \cdot \mathbf{v}_{t-1}$$

Step 2 — compute velocity using the gradient at the look-ahead position:

$$\mathbf{v}_t = \beta \cdot \mathbf{v}_{t-1} + \alpha \cdot \nabla L(\mathbf{w}_{\text{look-ahead}})$$

Step 3 — update the weight from the original position using this velocity:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \mathbf{v}_t$$

That's the entire difference. In Momentum, both the momentum push and the gradient push are computed from the same (current) point. In NAG, you first "look ahead" using momentum alone, evaluate the gradient there, and then make your actual move from the original point using that look-ahead gradient. This single change is what damps the oscillations.

5. Geometric Intuition

Momentum's behavior: Starting from a random point on the error surface, at every point you calculate two things: the momentum (initially zero) and the gradient. Early on, momentum is small, so movement is mostly gradient-driven. As you keep moving in a consistent direction, momentum builds up and starts dominating, pushing you faster and faster — until you overshoot the minimum. At that point, the gradient flips direction (now pointing backward), but the momentum is still large and initially dominates, so you keep moving forward briefly before turning back. This produces a back-and-forth oscillation pattern that gradually shrinks until you settle near the minimum.

NAG's behavior: Starting from the same point, you again track momentum and gradient. But now, before committing to a full jump, you first take a look-ahead jump using only the momentum term. At that look-ahead point, you check the gradient. If the gradient at the look-ahead point already points backward (i.e. you've look-ahead-jumped past where the slope reverses), your actual update will correct for this immediately, pulling you back a bit before you've even fully overshoot.

In effect: where Momentum would take a big, blind overshoot and have to fully reverse afterward, NAG takes a smaller, corrected overshoot — a kind of "early U-turn" — because it anticipates the slope ahead before committing fully. This is the central geometric intuition behind NAG: it looks ahead before deciding, and adjusts its decision-making accordingly.

6. Disadvantage of NAG

NAG works by damping the oscillations that occur in Momentum. While this is generally beneficial, it introduces a potential downside: **getting stuck in a local minimum**.

Imagine a 2D error surface where a ball is rolling down. With plain Momentum, the ball might accumulate enough momentum to cross over a small bump or shallow local minimum and continue toward a better (lower) minimum further along. But since NAG actively dampens momentum to reduce oscillation, the ball may not build up enough momentum to escape a local dip — it can oscillate briefly and then get trapped in a local minimum instead of reaching the global one.

This is the biggest known disadvantage of NAG, and on certain datasets or loss landscapes, it may require switching to a different optimizer to avoid this pitfall.

7. Keras Implementation

In Keras, Momentum and NAG are both implemented using the same **SGD** optimizer class — you simply change its parameter values:

Optimizer	Keras Configuration
Plain SGD	momentum = 0, nesterov = False
SGD with Momentum	momentum = <your β value>, nesterov = False

SGD with NAG	momentum = <your β value>, nesterov = True
---------------------	--

All three optimizers (plain SGD, Momentum, and NAG) can be implemented from this single SGD class in Keras simply by toggling these two parameters.

8. Summary

NAG is Momentum with one small but powerful tweak: instead of computing the gradient at your current position, you first take a look-ahead step using accumulated momentum, then compute the gradient at that look-ahead position to decide your actual move. This lets the optimizer anticipate the terrain ahead, reducing the oscillations and overshoot that plague plain Momentum, and generally leading to faster convergence. Its main drawback is an increased risk of getting trapped in local minima, since the dampened oscillations mean less momentum is available to "power through" shallow dips in the loss surface. In Keras, NAG is available directly from the SGD optimizer class by setting `nesterov=True` alongside a momentum value.

Reference: Video Timestamps

Source video: "Nesterov Accelerated Gradient (NAG) Explained in Detail | Animations | Optimizers in Deep Learning" (CampusX)

00:00 — Intro

00:47 — What is NAG?

09:12 — Mathematical Intuition of NAG

12:52 — Momentum (recap)

17:59 — Geometric Intuition of NAG

24:39 — Disadvantages

25:55 — Keras Code Implementation

27:30 — Outro

Notes compiled for personal study based on the CampusX YouTube lecture on Nesterov Accelerated Gradient. For the full explanation with animations, refer to the original video.